Robotics first assignement

Matteo Padrin 2199438

GitHub `github.com/matteopadrin15-beep/RoboticsProject`

How to run the code

Dependencies

All dependencies are installable via `pip`: `pip install numpy sympy matplotlib`

Library	Version tested	Purpose
`numpy`	≥ 1.24	Numerical computation, matrix operations
`sympy`	≥ 1.12	Symbolic Jacobian computation and lambdification
`matplotlib`	≥ 3.7	Trajectory plotting

How to run the code

```
python robtics.py
```

Simply run the code, and it will open all the windows needed for the assignment

Part 1

Since we are dealing with a robotic arm, the angles α and $2\pi + \alpha$ are equivalent in the final position, therefore in this solution we restrict to $\theta_i \in [-\pi, \pi), i \in \{1, 2, 3\}$.

To find the angles $\vec{\theta} = [\theta_1\, \theta_2\, \theta_3]^\top$, we need to solve the following system

$$\begin{cases} \cos(\theta_1) + \cos(\theta_1 + \theta_2) + \cos(\theta_1 + \theta_2 + \theta_3) = 1 \\ \sin(\theta_1) + \sin(\theta_1 + \theta_2) + \sin(\theta_1 + \theta_2 + \theta_3) = 2 \\ \theta_1 + \theta_2 + \theta_3 = \dfrac{\pi}{2} \end{cases}$$

Therefore from the last equation, we can simplify the model to

$$\begin{cases} \cos(\theta_1) + \cos(\theta_1 + \theta_2) = 1 \\ \sin(\theta_1) + \sin(\theta_1 + \theta_2) + 1 = 2 \\ \theta_1 + \theta_2 + \theta_3 = \dfrac{\pi}{2} \end{cases}$$

By squaring both sides and summing the first two equations, we get

$$1 + 1 + 2\Big(\cos(\theta_1)\cos(\theta_1 + \theta_2) + \sin(\theta_1)\sin(\theta_1 + \theta_2) \Big) = 2$$

Which implies that

$$\cos(\theta_1)\cos(\theta_1 + \theta_2) = -\sin(\theta_1)\sin(\theta_1 + \theta_2) \tag{$\star$}$$

This is equivalent to $\tan(\theta_1)\tan(\theta_1 + \theta_2) = -1$ only if the two cosines are not equal to zero, therefore there are three (+ 1) subcases starting from $\star$: the first one is if neither of them is different form zero, if $\cos(\theta_1) = 0$ and the other two arise when $\cos(\theta_1 + \theta_2) = 0$.

Case 1: $\cos(\theta_1) = 0$

$\theta_1 = \frac{\pi}{2}$ and $\theta_2 + \theta_3 = 0$. Substituting:

$$\begin{cases} \cos\left(\frac{\pi}{2} + \theta_2\right) = 1 \implies -\sin\theta_2 = 1 \\ \sin\left(\frac{\pi}{2} + \theta_2\right) = 1 \implies \cos\theta_2 = 0 \end{cases} \implies \theta_2 = -\frac{\pi}{2}, \quad \theta_3 = \frac{\pi}{2}$$

$$S_1 = \left[\frac{\pi}{2}, -\frac{\pi}{2}, \frac{\pi}{2}\right]^\top$$

Case 2: $\cos(\theta_1 + \theta_2) = 0$ with $\theta_1 + \theta_2 = +\pi/2$

$\cos(\pi/2) = 0$, $\sin(\pi/2) = 1$, and $\theta_3 = 0$. The system gives:

$$\begin{cases} \cos\theta_1 + 0 = 1 \\ \sin\theta_1 + 1 = 1 \end{cases} \implies \theta_1 = 0, \quad \theta_2 = \frac{\pi}{2}, \quad \theta_3 = 0$$

$$S_2 = \left[0, \frac{\pi}{2}, 0\right]^\top$$

Case 3: $\cos(\theta_1 + \theta_2) = 0$ with $\theta_1 + \theta_2 = -\pi/2$

$\cos(-\pi/2) = 0$, $\sin(-\pi/2) = -1$, and $\theta_3 = \pi$. The second equation becomes:

$$\sin\theta_1 + (-1) + \sin\left(\frac{\pi}{2}\right) = 2 \implies \sin\theta_1 = 2 \implies \nexists\, \theta_1 \in \mathbb{R}$$

No solution.

Case 4: everything goes well

Using $\tan(a+b) = \frac{\tan a + \tan b}{1 - \tan a \tan b}$:

$$\tan(\theta_1)\frac{\tan\theta_1 + \tan\theta_2}{1 - \tan\theta_1\tan\theta_2} = -1$$

Which means solving the quadratic equation

$$\tan^2\theta_1 + \cancel{\tan(\theta_1 + \theta_2)} = -1 + \cancel{\tan(\theta_1 + \theta_2)} \implies \nexists\, \theta_1 \in \mathbb{R}$$

.

No solution.

Hence, there are only two solutions:

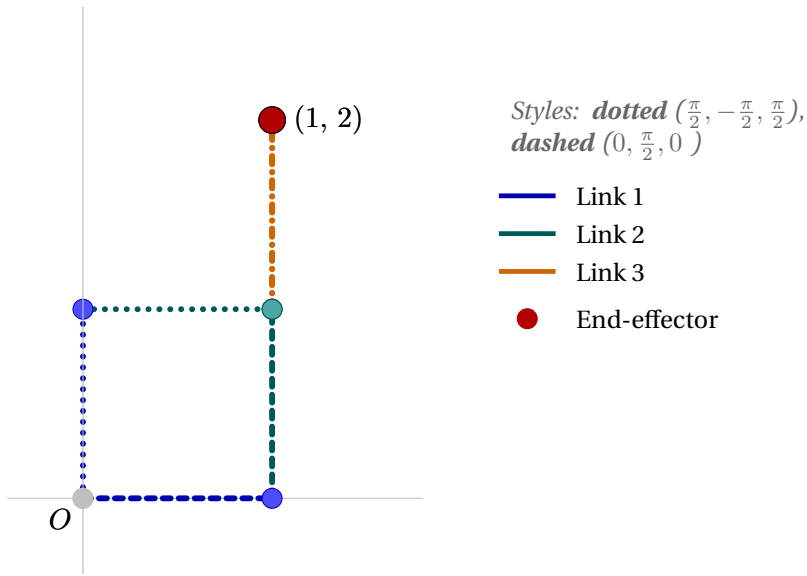

Figure 1: This picture displays the two solutions (they both reach the point $(1, 1)$ but follow two different paths (dotted and dashed), from there they reach point $(1, 2)$ (dash dotted)

To implement the methods we follow the formulas

$$q^{k+1} = q^k + \alpha J_\kappa^T(q^k)[x_e^d - \kappa(q^k)] \qquad \text{(Gradient)}$$
$$q^{k+1} = q^k + \alpha J_\kappa^{-1}(q^k)[x_e^d - \kappa(q^k)] \qquad \text{(Newton)}$$

Then we can use `sympy` python library to compute the $J_\kappa^{-1}(q^k)$ using the following lines to set up the basic variables:

```
from sympy import sin, cos, Matrix, lambdify
from sympy.abc import alpha, beta,gamma ,kappa
import numpy as np

J = Matrix([ cos(alpha) + cos(alpha+beta) + cos(alpha + beta +
    gamma) , sin(alpha) + sin(alpha+beta) + sin(alpha + beta + gamma
    ), alpha + beta + gamma])
Y = Matrix([alpha, beta, gamma])
Jac = J.jacobian(Y)
Jinv = Jac**-1 # Inversa
jinv_func = lambdify((alpha, beta, gamma), Jinv, 'numpy')
jac_direct_func= lambdify((alpha, beta, gamma), Jac, 'numpy')
```

After that the code follows directly where `history` is just used for plotting the values and α, β, γ are used as $\theta_1, \theta_2, \theta_3$.

```
def newton():
    q = np.array(INITIAL, dtype=float)
    target_pos = np.array(TARGET)
    history=[]
    for i in range(MAX_ITER):
        current_pos = kappa(q=q)
        history.append(current_pos[:2])
        error_vector = target_pos - current_pos
        if np.linalg.norm(error_vector) < TOL:
            return history
        inv_j_num = jinv_func(q[0], q[1], q[2])
```

```
        q = q + ALPHA * (inv_j_num @ error_vector)
    return history
```

```
def gradient(alpha=ALPHA):
    q = np.array(INITIAL, dtype=float)
    target_pos = np.array(TARGET)
    history=[]
    for i in range(MAX_ITER):
        current_pos = kappa(q=q)
        history.append(current_pos[:2])
        error_vector = target_pos - current_pos

        if np.linalg.norm(error_vector) < TOL:
            return history
        j_num = jac_direct_func(q[0], q[1], q[2])
        q = q + alpha * (j_num.T @ error_vector)
    return history
```

Then, if we plot it:

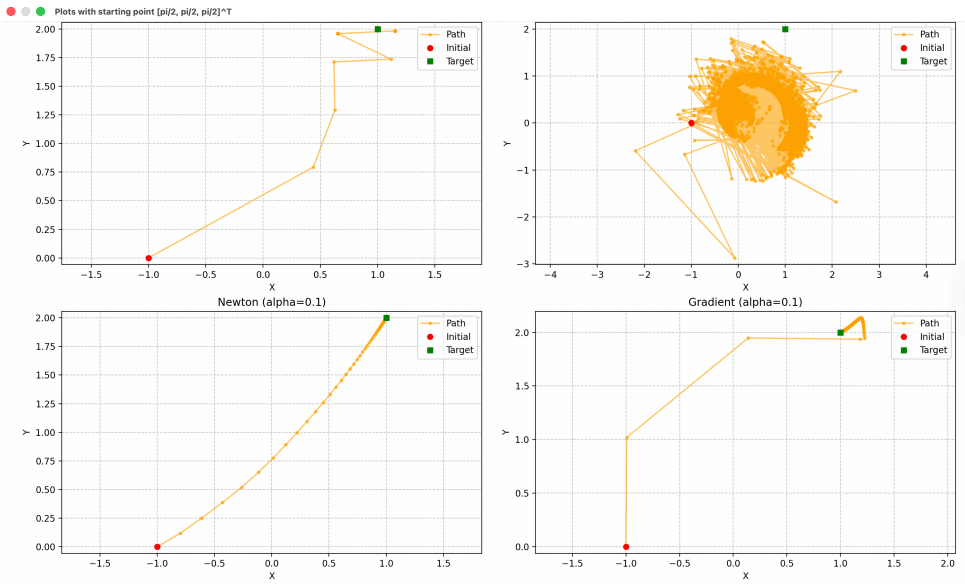

Figure 2: Results of the two methods with different values of α. In this plot we used $q^0 = [\frac{\pi}{2}, \frac{\pi}{2}, \frac{\pi}{2}]^\top$. In this configuration, we can see the importance of α since the stepsize actually determines the difference between convergence ($\alpha = 0.1$) or bouncing back and forth ($\alpha = 0.5$), of course choosing a large stepsize helps in a rapid convergence, but when "close" this could kill the process (indeed, with $\alpha = 0.5$ the gradient method does not converge at all even after one million iterations). As done in many deep learning algorithms, a possible choice would be to choose a function for the stepsize $\alpha(n)$ where n is the number of the iteration so that $\alpha(0)$ is regionally big and $\alpha(n)$ being a strictly decreasing function. A small detail that is important to notice is the following: in the top left plot, also the Newton method oscillates between some values so also Newton (for $\alpha = 0.5$) does not converge.

The problem arises if $q^0 = [0, 0, 0]^\top$ since it is a singular configuration, in fact:

$$\det(J) = -\sin(\theta_1)\cos(\theta_1 + \theta_2) + \sin(\theta_1 + \theta_2)\cos(\theta_1) = 2\sin(\theta_2) \quad \text{(i)}$$

(i) See the code for the calculation using `sympy`

Therefore, Newton's method risks to diverge if not properly adapted to avoid singular configurations. Then, if $\dot{q}^\star$ is such that $v_e = J\dot{q}^\star$, also $\dot{q} = \dot{q}^\star + P\dot{q}_0$ is a solution where P is a projection matrix into the null space of J ($\mathcal{N}(J)$) and from algebra we know $P = I - J^\top(JJ^\top)^{-1}J$. We also need to model $\dot{q}_0 = k_0\left(\frac{\partial\omega(q)}{\partial q}\right)^\top$ and a typical ω function used to escaping singularitites is $\omega(q) = \sqrt{\det(J(q)J(q)^\top) + \varepsilon}$ where $\varepsilon \gtrapprox 0$. Then we need to modify the code listed above, otherwise the solution would be

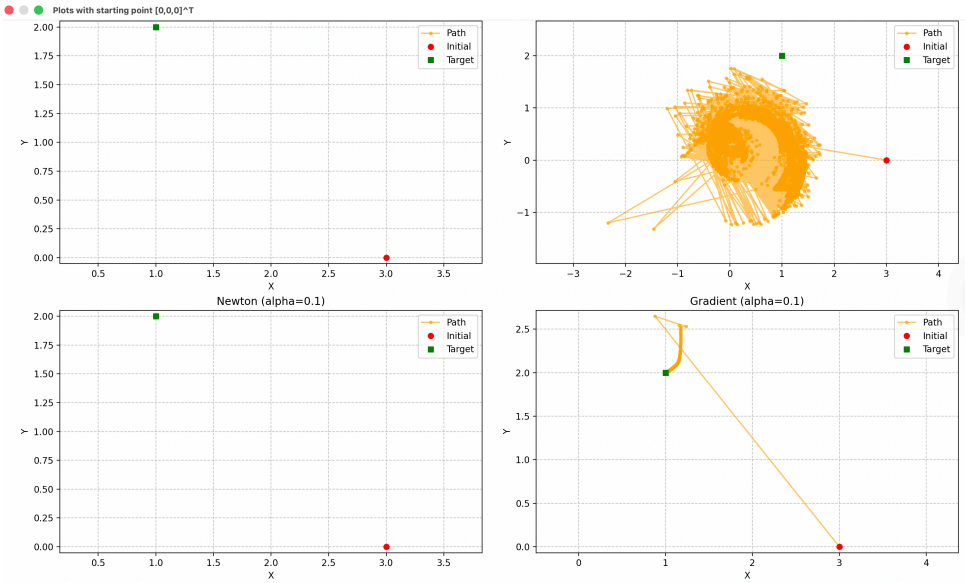

Figure 3: Results of the two methods with different values of α. In this plot we used $q^0 = [0, 0, 0]^\top$. Again we see the problem of the stepsize α.

This proves the issue: we can see that the left plots (the ones of the Newton's method) do not move the robotic arm from its initial position.

```
def redundantNewton(alpha_step=0.1, k0=0.5):
    w_sym = sqrt((Jac.T * Jac).det()+SCALAR_EPSILON) # w = root(det JJ )
    grad_w_sym = w_sym.diff(Y)
    w_func = lambdify((alpha, beta, gamma), w_sym, 'numpy')
    grad_w_func = lambdify((alpha, beta, gamma), grad_w_sym, 'numpy')
    jac_func = lambdify((alpha, beta, gamma), Jac, 'numpy')
    q = np.array(INITIAL, dtype=float)
    target_pos = np.array(TARGET)
    history = []

    for i in range(MAX_ITER):
        current_pos = kappa(q=q)
        history.append(current_pos.copy())
        error_vector = target_pos - current_pos
        if np.linalg.norm(error_vector) < TOL:
            print(f"Target raggiunto {i}")
            return history
        J_n = jac_func(q[0], q[1], q[2])
        g_w_n = grad_w_func(q[0], q[1], q[2])
        J_pinv = np.linalg.pinv(J_n, rcond=1e-3)
        I = np.eye(len(q))
```

```
        P = I - J_pinv @ J_n
        q_dot_task = J_pinv @ error_vector
        q_dot_null = P @ (k0 * g_w_n.T).flatten()
        q = q + alpha_step * (q_dot_task + q_dot_null)
    return history
```

We can finally see the results.

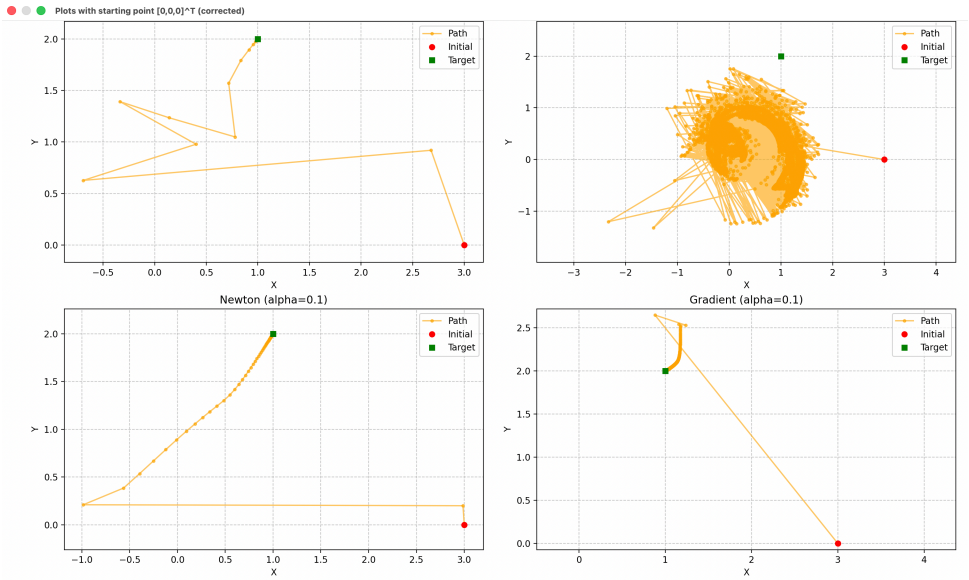

Figure 4: Results of the two methods with different values of α. In this plot we used $q^0 = [0, 0, 0]^\top$ and the `redundantNewton` method

A final possible attempt is using the generalized inverse (right pseudo inverse [ii]) $J^\dagger = J^\top (JJ^\top)^{-1}$ (again we need to be sure that $\det J^\dagger \neq 0$ hence we add λI for $\lambda \gtrapprox 0$). The code is really similar to the previous ones:

```
def generalizedNewton(alpha_step=0.1, lam=0.1):
    q = np.array(INITIAL, dtype=float)
    target_pos = np.array(TARGET)
    history = []
    i=0
    for i in range(MAX_ITER):
        current_pos = kappa(q=q)
        history.append(current_pos[:2].copy())
        error_vector = target_pos - current_pos
        if np.linalg.norm(error_vector) < TOL:
            return history,i
        J_n = jac_direct_func(q[0], q[1], q[2])
        JJT = J_n @ J_n.T
        J_d = J_n.T @ np.linalg.inv(JJT + lam * np.eye(3))
        q = q + alpha_step * (J_d @ error_vector)
    return history,i
```

The only issue is that computing the pseudo inverse (analytically) takes too much time for `sympy`, so I had to evaluate it using `numpy`, which is much faster (but just numerical).

[ii] Note that in the code below the calculations were done "manually" instead of using the `pinv` function from `numpy`

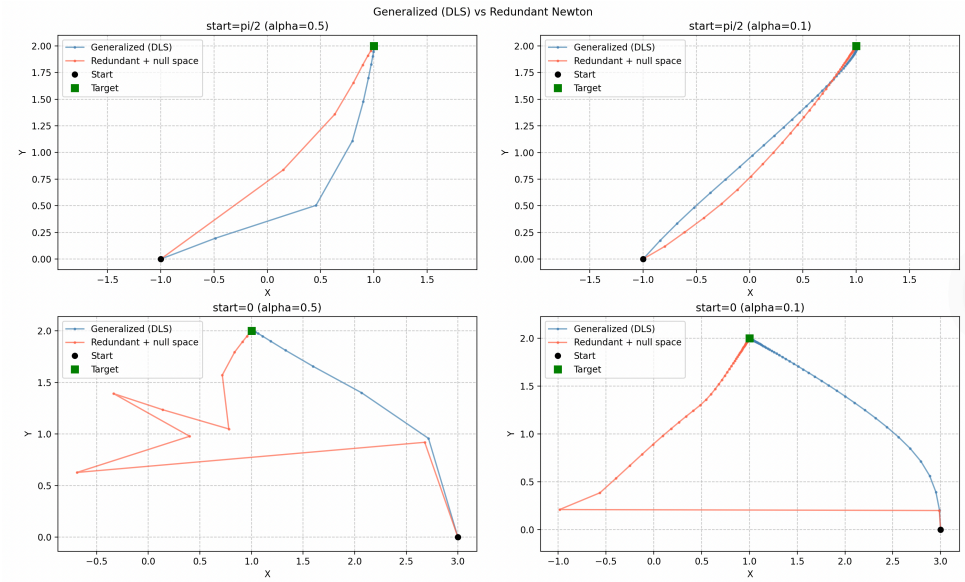

Figure 5: Results of the two methods with different values of α. In this plot we used $q^0 = [0, 0, 0]^\top$.

A couple of final considerations can be done: using the two methods on the given inputs, the number of iterations needed for the convergence are:

Start	**Alpha**	**Generalized**	**Redundant**
$\boldsymbol{\pi/2}$	0.5	23	17
$\boldsymbol{\pi/2}$	0.1	151	102
0	0.5	27	20
0	0.1	134	101

Overall, the generalized inverse performs worse in terms of the mere number of iterations, it is important to consider that the other path also does not consider other singularities that have to be avoided.

For the second task we can proceed just like the first one: then

$$\begin{cases} \cos(\theta_1) + \cos(\theta_1 + \theta_2) + \cos(\theta_1 + \theta_2 + \theta_3) = 1 \\ \sin(\theta_1) + \sin(\theta_1 + \theta_2) + \sin(\theta_1 + \theta_2 + \theta_3) = 2 \\ \theta_1 + \theta_2 + \theta_3 = -\dfrac{\pi}{2} \end{cases}$$

The difference is that the $\sin\left(-\frac{\pi}{2}\right)$ is -1, which means that the second equation becomes

$$\sin(\theta_1) + \sin(\theta_1 + \theta_2) + (-1) = 2$$

Hence, the sum of two sines is 3, which is not possible in the real numbers. Then this second configuration is impossible. This was previously calculated in the third case of the analytical solution, so this result comes with no surprise.

Moreover, I would need to reach the point $(1, 3)$ to have an orientation of $-\frac{\pi}{2}$ in $(1, 2)$, so the minimum overall length of the first two joints should be $\sqrt{10} \approx 3.1622$ and not 2.